## CS3001 - Computer Networks

# Assignment #3

Reliable Data Transfer Protocols

| | |
|---|---|
| **Name** | Ibrahim Johar Farooqi |
| **Student ID** | 23K-0074 |
| **Section** | BAI-6A |

## 1. Overview

This report documents the design, implementation, and testing of three Reliable Data Transfer (RDT) protocols over a simulated unreliable network channel. The protocols implemented are:

- rdt 3.0 – Stop and Wait
- Go-Back-N (GBN)
- Selective Repeat (SR)

The simulation is written in Python and models a uni-directional data transfer pipeline where control information (ACKs) flows in the opposite direction. The unreliable channel randomly introduces packet loss, corruption, and propagation delay.

## 2. Project Architecture

The project is organised into five Python modules:

| File | Responsibility |
|---|---|
| config.py | Central knob-panel: packet count, size, window size, timeout, loss/corrupt/delay probabilities, random seed. |
| network.py | Packet class (MD5 checksum, corruption simulation) and UnreliableChannel (loss, corruption, delay, per-run stats). |
| rdt30.py | rdt 3.0 Stop-and-Wait protocol — alternating-bit sender FSM and matching receiver FSM. |

| | |
|---|---|
| gbn.py | Go-Back-N sliding-window sender and cumulative-ACK receiver FSMs. |
| sr.py | Selective Repeat per-packet timer sender and buffering receiver FSMs. |
| main.py | CLI entry-point: protocol picker, scenario picker, unified execution, and summary timing. |

# 3. Protocol Implementations

## 3.1 rdt 3.0 – Stop and Wait

rdt 3.0 implements the alternating-bit protocol. The sender maintains two FSM states (WAIT_FOR_CALL and WAIT_FOR_ACK) and the receiver maintains two complementary states (WAIT_FOR_0 and WAIT_FOR_1).

**Sender behaviour:**

- Sends exactly one packet and blocks waiting for its ACK.
- On timeout, corrupt ACK, or wrong-sequence ACK: retransmits the same packet.
- Only advances the sequence bit (0 → 1 → 0) after a correct ACK.

**Receiver behaviour:**

- Accepts the packet whose sequence number matches the expected state.
- On corruption: sends an ACK for the opposite sequence number (implicit NAK).
- Duplicate packets (already-delivered sequence): re-ACKed and discarded.

The key design decision is the single in-flight packet constraint, which makes the implementation straightforward but limits throughput to one RTT per packet.

## 3.2 Go-Back-N (GBN)

GBN extends the pipeline to allow up to N unacknowledged packets in flight simultaneously using a single sliding window and a single window-level timer.

**Sender behaviour:**

- Fills the window [base, base+N) with new packets as slots become available.
- Maintains a single timer that starts when the first unACKed packet is sent.
- On timeout: retransmits all packets from base up to next_seq-1 (go back N).
- Processes cumulative ACKs — ACK k advances base to k+1.
- Ignores corrupt or duplicate ACKs; waits for the timer to recover.

**Receiver behaviour:**

- Accepts only in-order packets (seq_num == recv_seq).

- Out-of-order or corrupt packets are silently discarded.
- Re-sends the last good cumulative ACK so the sender knows the receiver position.

### 3.3 Selective Repeat (SR)

SR improves on GBN by maintaining individual per-packet timers and allowing the receiver to buffer out-of-order packets, avoiding unnecessary retransmissions.

**Sender behaviour:**

- Fills the window and starts an independent timer for each transmitted packet.
- On per-packet timeout: retransmits only that packet (not the whole window).
- Tracks individual ACKed flags; slides the window base only over consecutively ACKed packets.
- Corrupt ACKs are ignored; the per-packet timer handles recovery.

**Receiver behaviour:**

- Accepts any packet within [recv_base, recv_base+N) and buffers it.
- Sends an individual ACK for every correctly received packet.
- Delivers buffered packets in-order to the upper layer when the gap at recv_base is filled.
- Old packets (below the window) are re-ACKed so the sender can advance.
- Corrupt packets produce no ACK; the sender timer triggers retransmission.

## 4. Unreliable Channel Simulation

The UnreliableChannel class in network.py applies three independent random faults to every transmitted packet (data or ACK):

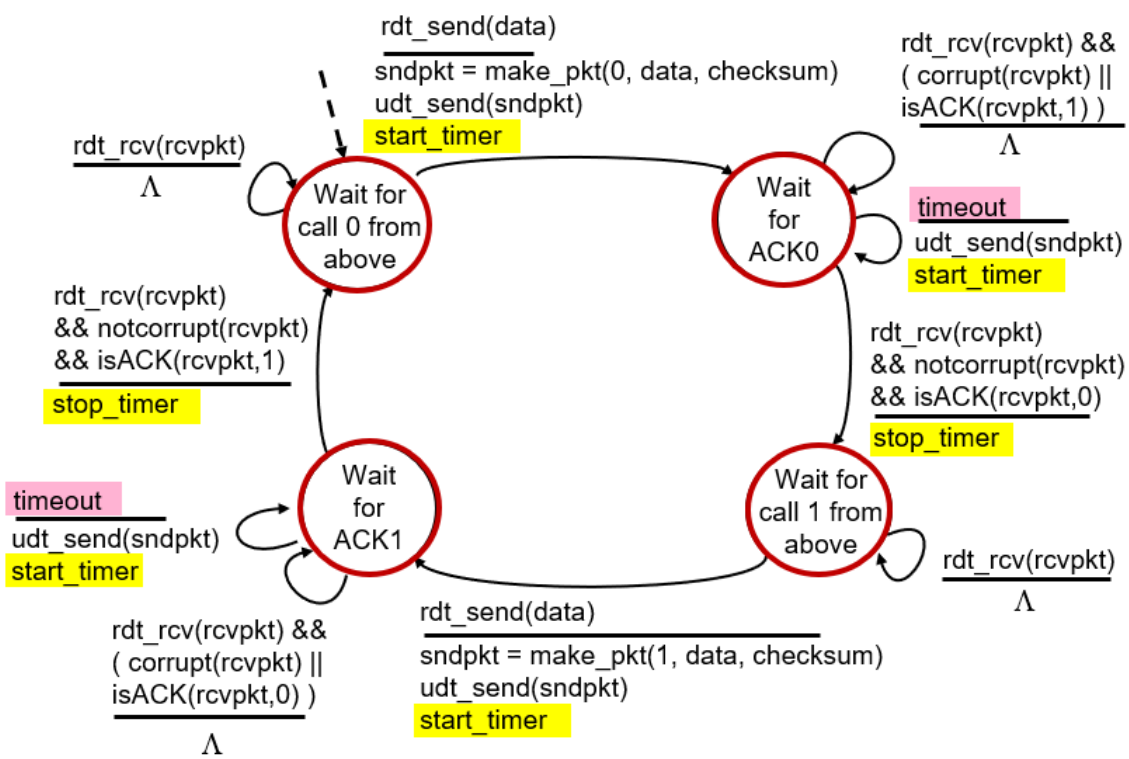
| Fault | Default Probability | Behaviour |
|---|---|---|
| Loss | 15% | Packet is silently dropped; transmit() returns None. |
| Corruption | 8% | Stored checksum is overwritten with a bogus value; is_corrupt() returns True. |
| Delay | 10% | Thread sleeps for a random duration in [0.01, DELAY_MAX] seconds. |

Integrity is verified via an MD5 checksum computed over the concatenation of seq_num, data, and is_ack. The channel always works on a deep copy of the packet so that the sender's original object is never mutated.

# 5. FSM Diagrams

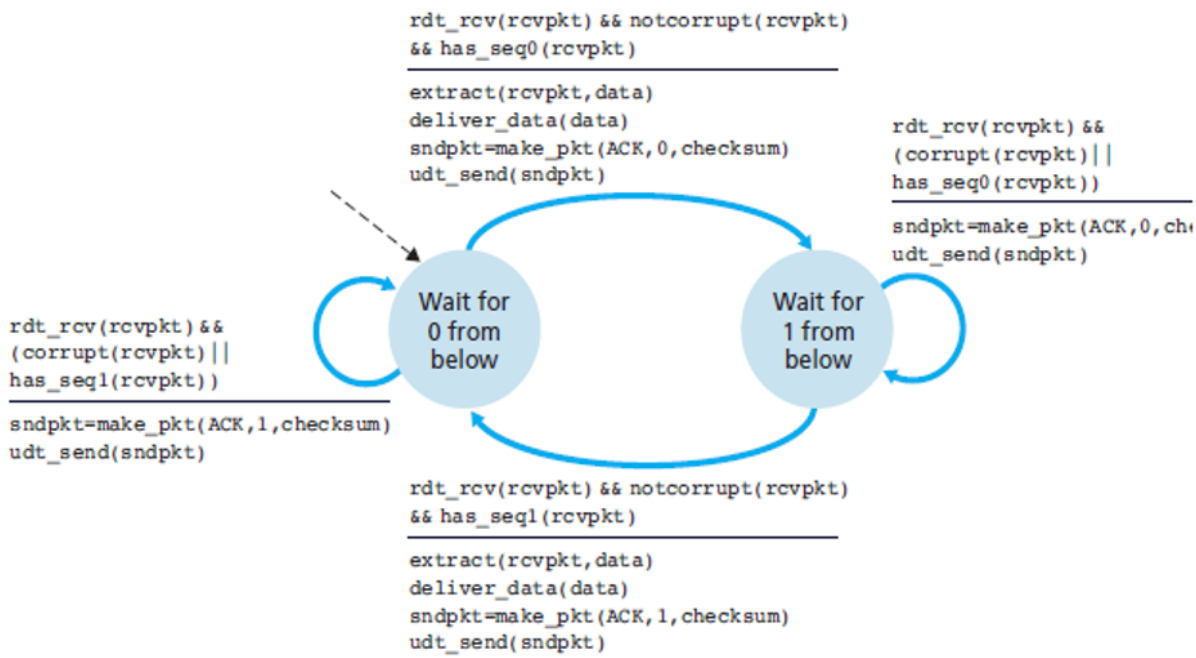
## 5.1 rdt 3.0 Sender FSM

| State | Event | Action | Next State |
|---|---|---|---|
| WAIT_FOR_CALL | rdt_send(data) | Make pkt(seq); udt_send(pkt); start timer | WAIT_FOR_ACK |
| WAIT_FOR_ACK | ACK(seq) — correct | Stop timer; flip seq bit | WAIT_FOR_CALL |
| WAIT_FOR_ACK | Timeout | Restart timer; udt_send(pkt) | WAIT_FOR_ACK |
| WAIT_FOR_ACK | Corrupt / wrong-seq ACK | Restart timer; udt_send(pkt) | WAIT_FOR_ACK |



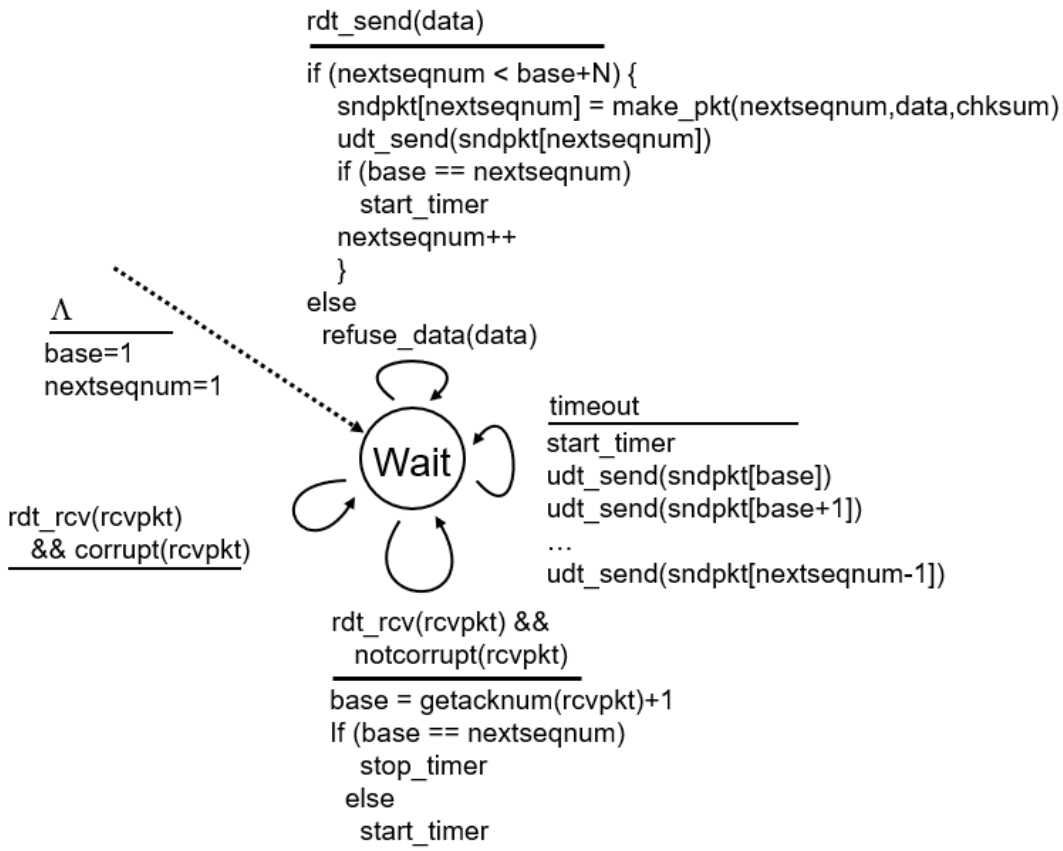
## 5.2 rdt 3.0 Receiver FSM

| State | Event | Action | Next State |
|---|---|---|---|
| WAIT_FOR_0 | pkt(0) — correct | deliver; send ACK(0) | WAIT_FOR_1 |
| WAIT_FOR_0 | pkt — corrupt | send ACK(1) [NAK] | WAIT_FOR_0 |
| WAIT_FOR_1 | pkt(1) — correct | deliver; send ACK(1) | WAIT_FOR_0 |
| WAIT_FOR_1 | pkt — corrupt | send ACK(0) [NAK] | WAIT_FOR_1 |



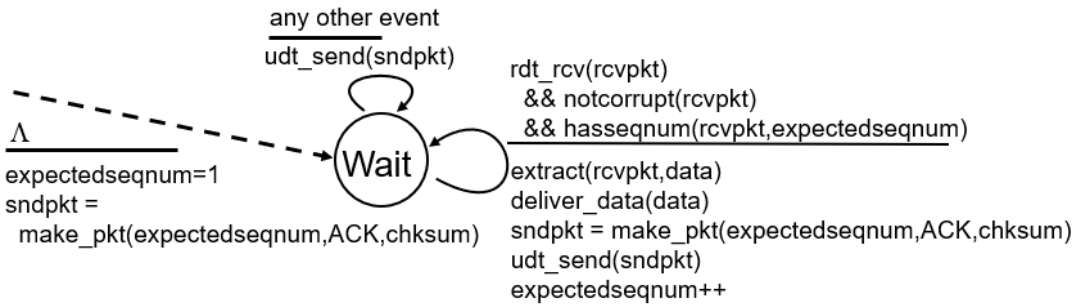

## 5.3 Go-Back-N Sender FSM

| Event | Action | Transition |
|---|---|---|
| Window slot free | Send pkt(next_seq); start timer if base==next_seq | Stay in SEND state |
| Cumulative ACK(k) | Set base=k+1; restart or stop timer | Stay in SEND state |
| Timeout | Retransmit pkt[base..next_seq-1]; restart timer | Stay in SEND state |
| Corrupt / old ACK | Ignore | Stay in SEND state |



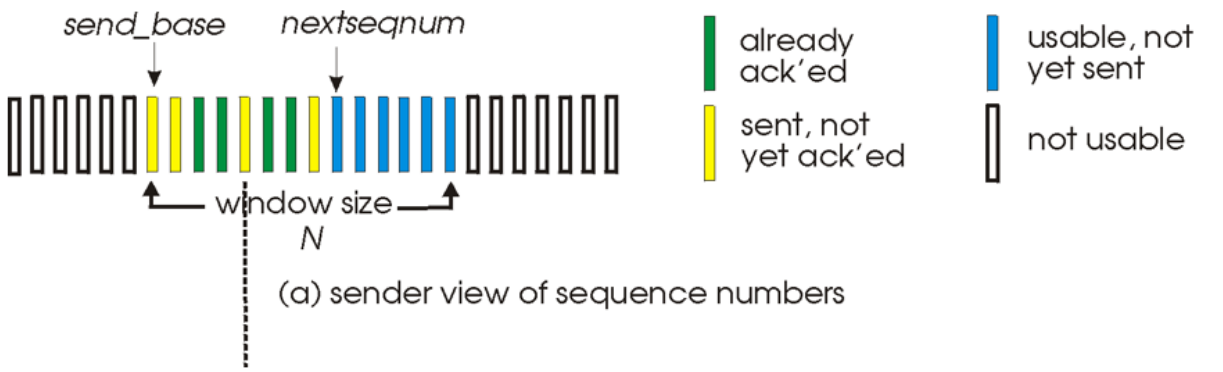

## 5.4 Go-Back-N Receiver FSM

| Event | Action | Transition |
|---|---|---|
| pkt(recv_seq) — correct | deliver; send ACK(recv_seq); recv_seq++ | Stay |
| pkt — out of order | Re-send ACK(recv_seq-1) | Stay |
| pkt — corrupt | Re-send ACK(recv_seq-1) if recv_seq > 0 | Stay |

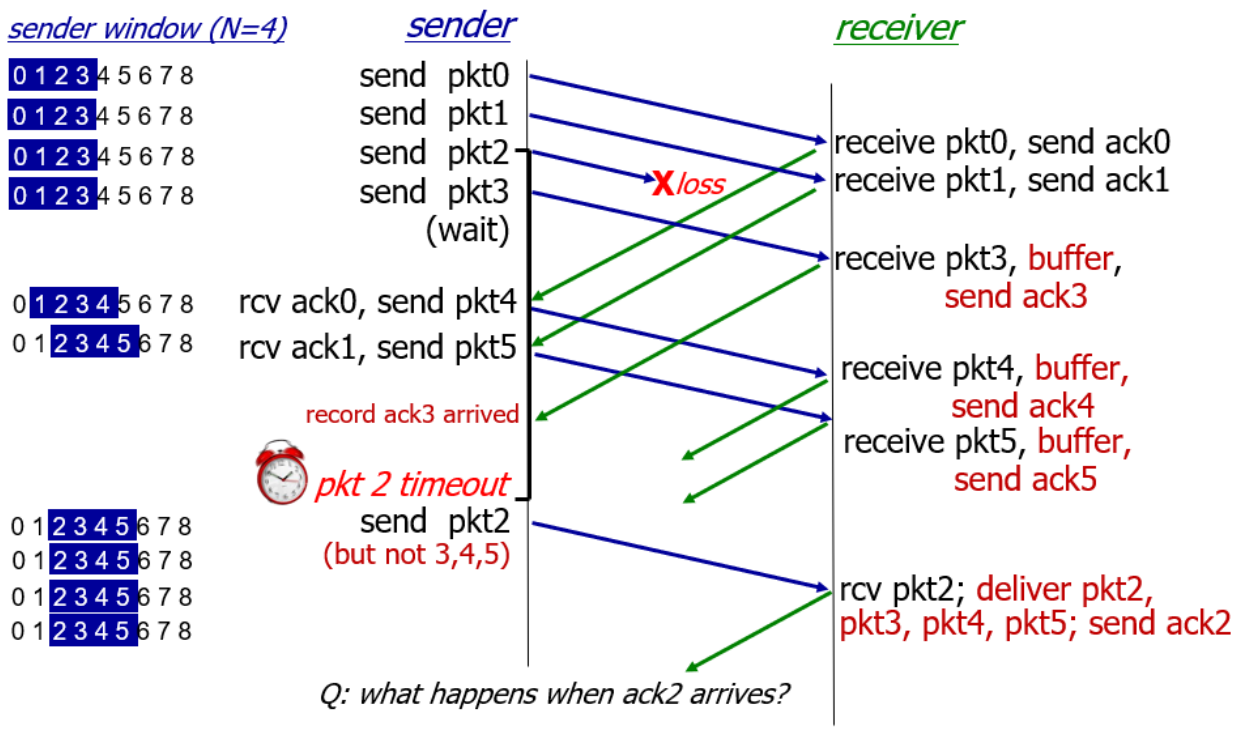


## 5.5 Selective Repeat Sender FSM

| Event | Action | Transition |
|---|---|---|
| Window slot free | Send pkt(next_seq); start individual timer[next_seq] | Stay |
| Individual ACK(k) | Mark acked[k]=True; cancel timer[k]; slide base while acked[base] | Stay |
| Timeout for pkt i | Retransmit only pkt[i]; restart timer[i] | Stay |
| Corrupt ACK | Ignore | Stay |




(a) sender view of sequence numbers



### 5.6 Selective Repeat Receiver FSM

| Event | Action | Transition |
|---|---|---|
| pkt(sn) in window — correct | Buffer pkt; send ACK(sn); deliver consecutive from recv_base; advance recv_base | Stay |
| pkt(sn) below window | Re-send ACK(sn) | Stay |
| pkt — corrupt | Discard silently (no ACK) | Stay |
| pkt(sn) above window | Discard | Stay |

## 6. Testing Scenarios & Results

Five pre-defined scenarios are available through the CLI. All tests used PACKET_COUNT=10, WINDOW_SIZE=4, TIMEOUT=2.0s, and RANDOM_SEED=7 for reproducibility.

| # | Scenario | Loss | Corrupt | Delay | Observed Behaviour |
|---|---|---|---|---|---|
| 1 | No errors (ideal) | 0% | 0% | 0% | All packets delivered, zero retransmissions. |
| 2 | Packet loss only | 30% | 0% | 0% | Timeouts trigger retransmissions; all packets eventually delivered. |
| 3 | Corruption only | 0% | 30% | 0% | Checksum detection triggers retransmissions; delivery guaranteed. |
| 4 | Delayed packets only | 0% | 0% | 50% | Delays may trigger spurious timeouts; |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | duplicate ACKs handled correctly. |
| 5 | All errors combined | 15% | 8% | 10% | All three protocols recover and deliver all packets; SR has fewest retransmissions. |

Key observations across protocols:

- rdt 3.0 has the highest total simulation time due to one RTT per packet. In a 15% loss scenario it retransmits approximately 1–2 packets on average.
- GBN retransmits whole windows on timeout, so it incurs more redundant retransmissions than SR but less than rdt 3.0 in high-loss conditions due to pipelining.
- SR shows the best performance under mixed error conditions because only lost or corrupted packets are retransmitted.

## 7. How to Run

Prerequisites: Python 3.8+ (no third-party libraries required).

1. Clone / unzip the project folder.
2. (Optional) Edit config.py to adjust parameters.
3. Run the entry-point:

```
python main.py
```

4. When prompted, choose a protocol (1–4) and a test scenario (1–5).
5. 5. To run all three protocols under the same scenario, choose option 4.

## 8. Configurable Parameters

| Parameter | Default | Description |
|---|---|---|
| PACKET_COUNT | 10 | Total number of data packets to send. |
| PACKET_SIZE | 8 bytes | Payload size in bytes per packet. |
| WINDOW_SIZE | 4 | Sliding window size N for GBN and SR. |
| TIMEOUT | 2.0 s | Retransmission timeout period. |
| LOSS_PROB | 0.15 | Probability of a packet being dropped. |
| CORRUPT_PROB | 0.08 | Probability of a packet being corrupted. |
| DELAY_PROB | 0.10 | Probability of a packet being delayed. |
| RANDOM_SEED | 7 | Fixed RNG seed for reproducible runs. Set to None for randomness. |

## 9. Conclusion

All three reliable data transfer protocols were successfully implemented and tested against five distinct channel conditions. The simulation confirms the theoretical properties described in the Kurose & Ross textbook: rdt 3.0 is simple but slow, GBN improves throughput via pipelining at the cost of bulk retransmissions on errors, and Selective Repeat achieves the best efficiency by retransmitting only what is necessary.

The modular design (separate config, network, and protocol modules) makes it straightforward to extend the simulation with additional protocols or more sophisticated channel models.